

0.89.1. La fonction `teindre()` — une case dans une palette

`#include <teindre>` : `teindre(colonne, ligne, palette)` met une case du fond dans une autre palette (Game Boy Color).

1. Ce qu'il faut comprendre

Ce qu'elle fait : toute case est dans la **palette 0**, jusqu'à ce qu'on la **teigne**. `teindre(colonne, ligne, palette)` met cette case dans une autre palette : la même tuile peut avoir des couleurs différentes d'une case à l'autre.

La ligne à écrire : `#include <teindre>`.

Ses arguments : la colonne, la ligne, la palette (0 à 7). `palette` | DEVANT fait passer la case devant les personnages.

Ce qu'elle coûte : environ **100 octets** la première fois.

Essaie : teins aussi la case (7, 8) : deux lettres en bleu.

On retrouve `teindre` dans 15 leçons

[0.89.2](#), [0.92](#), [0.92.1](#), [0.92.2](#), [36](#), [37](#), [102](#), [103](#), [105](#), [106](#), [110.6](#), [110.7](#), [110.9](#), [110.13](#), [152](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **310 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         47 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
CopierTuiles   58 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         64 o ← CopierTuiles – 4 lettres sur 43
"BLEU"         4 o ← texte() – un texte gravé
```

Lettres gravées : BELU

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
teindre()      × 1    17 o sur place, dans main()
couleurFond() × 1    12 o sur place, dans main()
```

```
texte()      x 1    11 o sur place, dans main() + EcrireTexte (10 o)
image()      x 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <teindre> : une case dans une palette ----
// BLEU s'écrit ; seule sa première case passe dans la palette 2, bleue.

#include <couleurFond>
#include <texte>
#include <teindre>      // teindre() : la palette d'une case
```

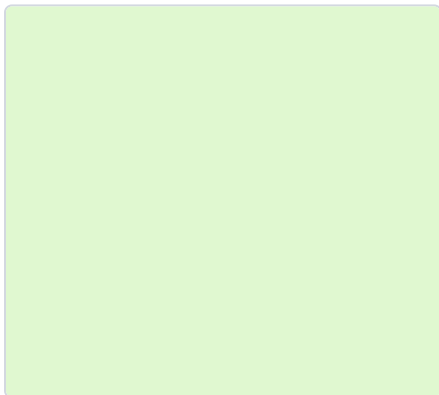
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    couleurFond(2, 3, 0, 0, 31); // palette 2, teinte 3 (les lettres) : bleu
    texte(6, 8, "BLEU");
    teindre(6, 8, 2);             // la case du B dans la palette 2

    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

BLEU, dont seul le B est bleu.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la case du B est dans la palette 2 (2)

✓ la case du L reste dans la palette 0

Ensuite : **0.89.2. Chaque mot sa couleur : texteCouleur — un arc-en-ciel** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++